App Dev Project Report

1. Student Details

Name: Biswajit Kar
Roll Number: 23f100253
Email: 23f1001253@ds.study.iitm.ac.in
About Me: I am a student interested in web application development. I enjoy building real-world applications using Python and Flask. This project helped me understand backend development, database design, and full-stack web application architecture.

---

2. Project Details

Project Title: Placement Portal Application

Problem Statement:

Many colleges manage placement activities manually, which makes it difficult to track student applications, company drives, and selection status.

The objective of this project is to develop a **web-based placement portal** that allows **students to apply for placement drives, companies to create job drives, and administrators to manage the entire placement process efficiently.**

Approach:

The application is developed using the **Flask web framework** with **SQLAlchemy ORM** for database interaction.

The system contains **three main user roles**:

1. **Admin**
2. **Company**
3. **Student**

The workflow of the system:

- Students register and upload their resume.
- Companies register and wait for admin approval.
- Admin approves companies and students.
- Companies create placement drives.
- Students apply for drives.
- Companies review applications and update the selection status.

The system also prevents duplicate applications and maintains application history.

---

3. AI/LLM Declaration

I used ChatGPT (LLM) only for assistance in implementing **secure file upload handling** in the Flask application, particularly for understanding the use of the `secure_filename` function from the `werkzeug` library.

The overall usage of AI tools was minimal (approximately **5–10%**) and limited to learning support and small code suggestions.

All core implementation, database design, routing logic, debugging, and integration of the placement portal system were completed independently.

---

4. Technologies and Frameworks Used

Technology	Purpose
Flask	Backend web framework
SQLAlchemy	ORM for database interaction
SQLite	Database for storing application data
Jinja2	Template engine for dynamic HTML

Bootstrap 5	Frontend user interface
Werkzeug	Secure file upload handling
Python	Core programming language

---

5. Database Schema / ER Diagram

Tables

1. **User**

Stores login credentials.

Fields:

- id
- username
- email
- password
- role (admin / student / company)

---

2. **Student**

Stores student profile information.

Fields:

- id
- user_id
- name
- email
- department
- resume
- approval_status

---

3. **Company**

Stores company details.

Fields:

- id
- user_id
- name
- email
- hr_contact
- website
- approval_status

---

4. **Drive**

Stores placement drive information.

Fields:

- id
- company_id
- name
- job_title
- job_description
- salary
- location
- eligible_criteria
- application_deadline
- status

---

5. **Application**

Stores student applications for drives.

Fields:

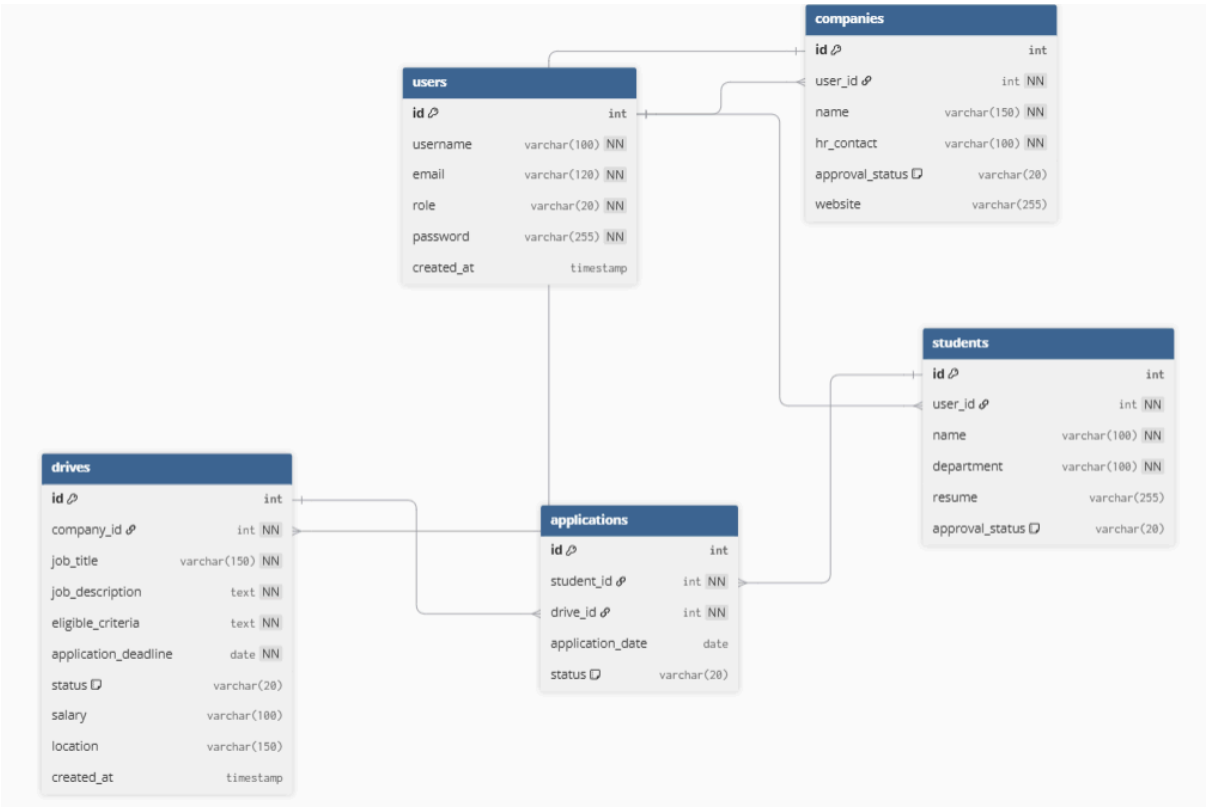
- id
- student_id
- company_id
- drive_id
- application_date
- status

Relationships

- **User** → **Student** (One to One)
- **User** → **Company** (One to One)
- **Company** → **Drive** (One to Many)
- **Student** → **Application** (One to Many)

- **Drive → Application** (One to Many)

link:
https://dbdiagram.io/d/placement-portal-69972289bd82f5fce2331b07



(example taken form dbdiagram.io)

---

6. API Resource Endpoints

Endpoint	Method	Description
/login	GET/POST	User login
/register	GET/POST	Student/Company registration
/admin	GET	Admin dashboard
/company/	GET	Company dashboard
/student/	GET	Student dashboard
/create_drive	POST	Company creates placement drive
/apply/<s_id>/<d_id>	GET	Student applies to drive

/action/	POST	Company updates application status
/approve/	GET	Admin approves company
/blacklist_student/	GET	Admin blacklists student

---

7. Architecture and Features (optional)

Architecture Overview

The project follows a **Flask-based modular structure** where different components handle specific responsibilities in the web application.

Project Structure:

```
project/
│
├── app.py          – Main Flask application entry point and route controller
│
├── application/     – Contains database models and core backend logic
│
├── instance/        – Stores configuration files and local database (SQLite)
│
├── templates/       – Jinja2 HTML templates used to render dynamic web pages
│
├── static/
│   └── resumes/    – Stores uploaded student resume files (PDF)
```

Component Description

- **app.py**
  This is the main entry point of the Flask application. It defines routes, handles HTTP requests, manages user authentication, and controls the overall workflow of the

placement portal.

- **application/**
  This directory contains the database models and core backend logic used to interact with the database through SQLAlchemy.

- **instance/**
  This folder stores instance-specific files such as the SQLite database and configuration settings used by the Flask application.

- **templates/**
  Contains HTML templates built using the **Jinja2 template engine** to dynamically display data such as dashboards, drive details, and application status.

- **static/resumes/**
  Stores uploaded student resumes securely. Resume files are uploaded during student registration and profile updates.

Implemented Features:

Core features implemented:

✔ User authentication system
✔ Role-based login (Admin, Company, Student)
✔ Student registration with resume upload
✔ Company registration and admin approval
✔ Admin dashboard to manage users and drives
✔ Company placement drive creation
✔ Student application system
✔ Application status tracking
✔ Blacklisting system for companies and students
✔ Placement history tracking

---

8. Video Presentation

Drive Link:
https://drive.google.com/file/d/1dL0RNMQvNhgUORzcW6lrEk-XamTtYgA-/view?usp=drivesdk

---